

Exercise 12: Reinforcement And Efficient Learning

Task 1:

In the context of tabular Reinforcement Learning, the phenomenon known as "State Explosion" represents a fundamental limitation when moving from simple grid worlds to more realistic environments.

- Explain why classical tabular Q-Learning fails when applied to an environment like LunarLander. In your explanation, refer explicitly to its 8-dimensional continuous state space (coordinates, velocities, angles, etc.) and describe what would happen if you tried to discretize these features into a standard Q-table.
- Explain how Deep Q-Networks (DQN) structurally and mathematically resolve this limitation. How does the function approximation of a neural network replace the traditional lookup table?
- Even though DQN utilizes a deep neural network, it is strictly classified as a Value-based method. Explain the fundamental difference between Value-based and Policy-based reinforcement learning. What does a DQN actually output compared to a Policy-gradient network, and how does this dictate the agent's action selection?

Task 2:

In game theory and real-world environments, Reinforcement Learning agents face different types of action spaces and strategic requirements.

- Explain why a Value-based agent (like classical Q-Learning or DQN) is fundamentally incapable of learning a successful strategy for a game like Rock-Paper-Scissors when playing against a human opponent who analyzes patterns.
- Continuous spaces often pose a significant challenge in reinforcement learning. Clarify the distinct difference between how Deep Q-Networks (DQN) handle continuous *state* space versus why they still fail when dealing with continuous *action* spaces (e.g., outputting an exact fluid steering angle like -12.345°).

Task 3:

Your goal is to optimize a convolutional neural network for CIFAR Dataset (<https://cave.cs.toronto.edu/kriz/cifar.html>) by adding regularization techniques and dynamic training controls. You will modify the model architecture, set up specific training behavior, and analyze how these changes impact performance.

Please complete the following steps in your notebook:

1. Modify the `build_cnn` Function:

- Expand the function signature to accept two new function parameters: `dropout_rate` (default `0.0`) and `l1_reg` (default `0.0`).
- **L_1 Regularization:** If `l1_reg > 0`, apply an L_1 kernel regularizer to each `Conv2D` layer inside the loop.
- **Dropout:** If `dropout_rate > 0`, insert a `Dropout` layer immediately after each `MaxPooling2D` layer inside the loop.

2. Configure Callbacks:

- Create an **EarlyStopping** callback that monitors validation loss ([val_loss](#)), stops training after a specific number of epochs of no improvement (i.e 5), and restores the best model weights.
- Create a **ReduceLROnPlateau** callback that monitors validation loss and reduces the learning rate by a factor (i.e. 0.5) if no improvement is seen for a specific number of epochs (i.e 3).

3. Train and Compare:

- different configurations for up to 30 or more epochs using your callbacks and compare the results, i.e:
- **Baseline:** [dropout_rate = 0.0](#), [l1_reg = 0.0](#)
- **Dropout Only:** [dropout_rate = 0.3](#), [l1_reg = 0.0](#)
- **Combined:** [dropout_rate = 0.3](#), [l1_reg = 0.0001](#)
- ...

Task 4:

Designing an effective reward function is widely considered one of the most challenging aspects of setting up a Reinforcement Learning system. Suppose a basic implementation of the **Snake** game uses the following straightforward reward rules:

- Snake eats an apple: +10
- Snake dies (hits a wall or collides with itself): -10
- Any other standard movement step: 0

To speed up the agent's learning process, a programmer decides to change the reward for "any other standard movement step" from 0 to +1, arguing that this explicitly rewards the snake for surviving longer.

Analyze what kind of unintended behavior (**Reward Hacking**) this alteration is likely to provoke from the agent.